



GEBZE TECHNICAL UNIVERSITY
Department of Computer Engineering
CSE 396 — Computer Engineering Project | Spring 2026

Module Interface & Header Design

AI-Powered Smart PPE Inspection Station

GROUP-11

Course: CSE 396 — Computer Engineering Project
Term: Spring 2026
Assignment Title : Assignment 2 : Module Interface & Header Design
Project Title: AI-Powered Smart PPE Inspection Station
Submission Date: 29.03.2026

Team Members

Student Name	Student ID	Assigned Module
Muhammed Emir Kara	210104004071	Module 2 (Turnstile) — Primary
Mümincan Durak	210104004057	Module 2 (Turnstile) — Primary
Hüseyin Elyesa Yeşilyurt	210104004080	Module 4 (Backend) — Primary
Alperen Söylen	220104004024	Module 3 (IoT Module) — Primary
Ahmet Emre Kurt	220104004016	Module 4 (Backend) — Primary
Zeynep Etik	220104004035	Module 1 (AI & Vision) — Primary
Emre İlhan Şenel	230104004907	Module 5 (UI & UX) — Primary
Tarık Saeede	200104004804	Module 5 (UI & UX) — Primary

1. Module–Student Matching Table

The table below shows the five modules of the AI-Powered Smart PPE Inspection Station, the students primarily responsible for each module, and a brief description of each module's role in the system.

Module ID	Module Name	Responsible Student(s)	Brief Role
MOD-01	AI & Vision Module	Zeynep Etik (Primary) Alperen Söylen (Secondary) Muhammed Emir Kara (Secondary)	Trains and deploys the YOLO-based PPE detection model on Raspberry Pi 5 with AI Kit; provides detect(frame) → list of detected items API.
MOD-02	Turnstile Module	Mümincan Durak (Primary) Muhammed Emir Kara (Primary) Ahmet Emre Kurt (Secondary)	Designs and fabricates the physical turnstile gate; implements servo motor control; exposes gate.open() / gate.close() API.
MOD-03	IoT Module	Alperen Söylen (Primary) Zeynep Etik (Secondary) Mümincan Durak (Secondary) Emre İlhan Şenel (Secondary) Hüseyin Elyesa Yeşilyurt (Secondary)	Hardware-software bridge: RFID reader integration, GPIO pin management, wiring, and camera pipeline setup; provides rfid.read_card() → card_id API.
MOD-04	Backend & Database Module	Ahmet Emre Kurt (Primary) Hüseyin Elyesa Yeşilyurt (Primary) Tarık Saeede (Secondary)	REST API server and relational database; handles worker registration, card lookup, role-based PPE requirement retrieval, access decision logic, and entry log writes.
MOD-05	UI & UX Module	Tarık Saeede (Primary) Emre İlhan Şenel (Primary)	Admin Panel (PC side) for worker registration and log viewing; Turnstile Display (tablet) for real-time camera feed, inspection results, and animated instructions.

1.1 Person–Module Assignment Matrix

Person	Module 1 (AI & Vision)	Module 2 (Turnstile)	Module 3 (IoT)	Module 4 (Backend)	Module 5 (UI & UX)
Zeynep	P		S		
Alperen	S		P		
Mümincan		P	S		
Emir	S	P			
Ahmet		S		P	
Elyesa			S	P	
Tarık				S	P

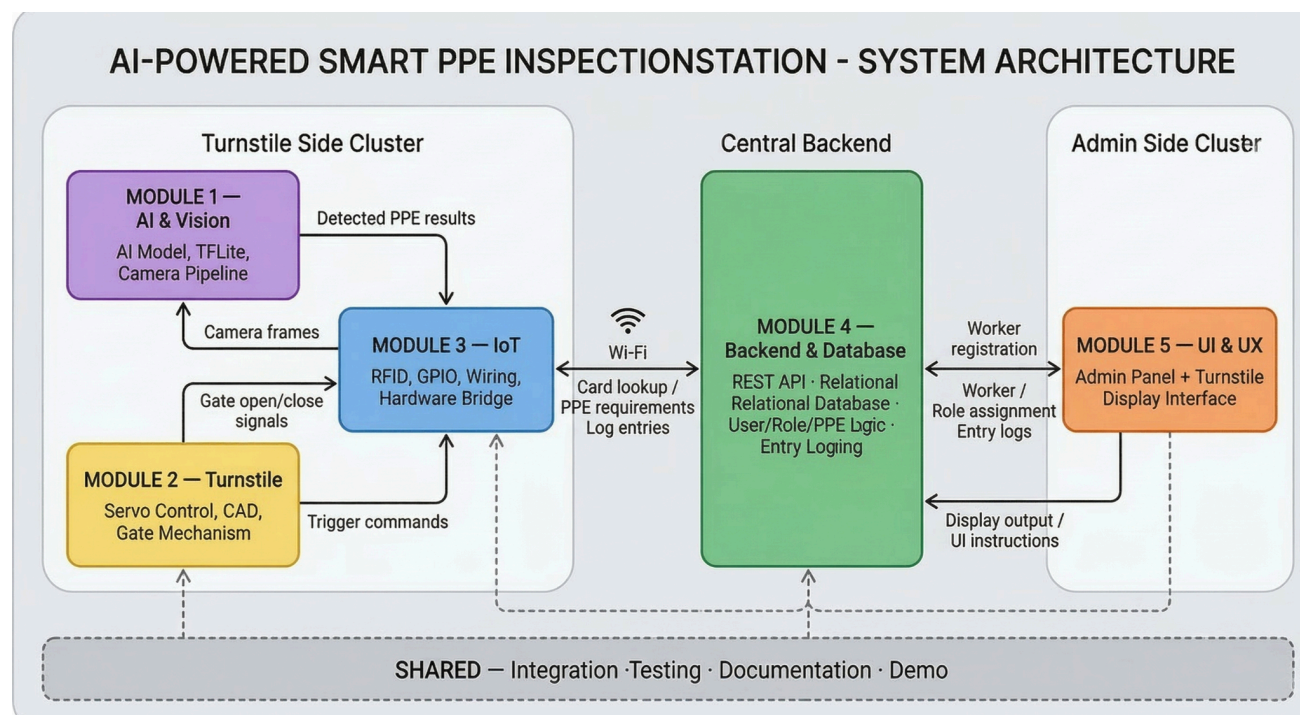
Person	Module 1 (AI & Vision)	Module 2 (Turnstile)	Module 3 (IoT)	Module 4 (Backend)	Module 5 (UI & UX)
Emre			S		P

P = Primary (main responsibility) S = Secondary (support)

2. System / Module Overview

2.1 Overall System Architecture

The AI-Powered Smart PPE Inspection Station is composed of two physical units — the Turnstile Side and the Admin Side — connected over a Wi-Fi LAN to a Central Backend Server. All communication uses HTTP REST over the local network; real-time display updates on the Turnstile Display are delivered directly from the Raspberry Pi. The system operates entirely offline without any cloud dependency.



2.2 Module Descriptions

MOD-01 — AI & Vision Module

Owned by Zeynep Etik and Alperen Söylen. This module is responsible for the entire machine-learning pipeline: dataset collection and augmentation from the Ultralytics Construction PPE Dataset (supplemented by Roboflow Universe and Kaggle), training a YOLOv8n object detection model, converting it to ONNX format with INT8 post-training quantization, and deploying it on the Raspberry Pi 5 AI Kit NPU. The module exposes a single Python API: `detect(frame: CameraFrame) → DetectionResult`, returning the set of detected PPE class labels (helmet, gloves, vest, boots, googles, no_helmet, no_gloves, no_vest, no_boots, no_googles, person).

MOD-02 — Turnstile Module

Owned by Mümincan Durak and Muhammed Emir Kara. This module covers the physical design and electromechanical control of the turnstile gate. The gate arm mechanism is designed in CAD and fabricated using 3D printing or acrylic/aluminium materials. Two high-torque servo motors driven by PWM signals from Raspberry Pi GPIO pins control the lock and rotation of the gate. The module exposes `gate.open()` and `gate.close()` commands consumed by the IoT module on receipt of an access decision.

MOD-03 — IoT Module

Support role shared by Mümincan Durak, Muhammed Emir Kara, and Alperen Söylen. This module is the hardware-software bridge on the Raspberry Pi 5. It manages the RC522 RFID reader (SPI interface), GPIO pin assignment for all peripherals, power distribution wiring, and camera pipeline setup. It orchestrates the main inspection state machine (IDLE → IDENTIFYING → INSPECTING → GRANTED/DENIED) and exposes `rfid.read_card() → str (card_id)` to the rest of the system.

MOD-04 — Backend & Database Module

Owned by Ahmet Emre Kurt and Huseyin Elyesa Yesilyurt. This module implements the central REST API server and relational database for the turnstile system. The database schema is centered on workers, roles, ppe_items, role_ppe_requirements, entry_logs, and detection_details. Worker records include RFID identity and role linkage; role-to-required-PPE mapping is modeled through a junction table (not a single required_ppe_list column). Entry logs store scanned RFID, result, timestamps, and related detection details, while missing PPE is derived in API responses. The server exposes endpoints for worker registration/management, RFID card lookup, role PPE retrieval, PPE catalog access, and entry-log writes/reporting. Inspection outcomes use PASS/FAIL/UNKNOWN_CARD semantics consistent with the API contract.

MOD-05 — UI & UX Module

Owned by Tarık Saeede and Emre İlhan Şenel. This module delivers two user-facing interfaces. The Admin Panel (React 18 web application) provides worker registration forms, RFID card assignment, role selection, worker list management, entry log viewing, and compliance statistics. The Turnstile Display (tablet application) shows a live camera feed with bounding boxes, animated instructions, and the ACCESS GRANTED / ACCESS DENIED screen with personalised worker name and missing-items list.

3. Hardware & Platform Summary

3.1 Target Hardware

#	Component	Model / Spec	Role in System
1	Raspberry Pi 5	16 GB RAM + AI Kit (Hailo-8L NPU)	Main controller; runs AI inference, IoT logic, display controller
2	RPi Camera Module V3	12 MP, wide-angle, CSI interface	Frame capture for PPE detection pipeline
3	RC522 RFID Reader	ISO 14443A, SPI interface — ×2 units	Card scanning on both turnstile and admin sides
4	RFID Cards / Tags	ISO 14443A cards — ×10	Worker identification tokens
5	Servo Motors	High-torque, PWM — ×2	Gate lock and rotation mechanism
6	Display / Tablet	7" touchscreen or HDMI monitor (TBD)	Turnstile-side inspection result display
7	Power Supply	LiPo battery + 5V/5A regulator	Powers Raspberry Pi, servos, and peripherals
8	MicroSD Card	Samsung Evo Plus 128 GB	OS and model storage on Raspberry Pi
9	Chassis / Frame	3D-printed or acrylic/aluminium	Physical turnstile structure housing all components
10	Servo Driver	PCA9685 16 Channel I2C PWM/Servo Driver Board	Translates I2C commands from the Raspberry Pi into PWM signals to move the servos.

3.2 Communication Buses

Bus / Protocol	Connected Devices	Notes / Constraints
SPI	RC522 RFID Reader ↔ Raspberry Pi 5 GPIO	RFID card read operations; clock frequency: TBD
CSI	RPi Camera Module V3 ↔ Raspberry Pi 5	Camera frame capture; 15–30 fps at 1080p or lower resolution for inference
PWM (GPIO)	Raspberry Pi 5 GPIO ↔ Servo Motors (×2)	Gate open/close control; PWM frequency: TBD (typ. 50 Hz)
HDMI / DSI	Raspberry Pi 5 ↔ Display	Display output for turnstile UI; interface type TBD
Wi-Fi 802.11ac	RPi 5 ↔ Backend Server ↔ Admin Panel PC	HTTP REST; Turnstile Display updated directly from RPi

3.3 Timing Constraints

Parameter	Value / Target	Notes
Camera Frame Rate	15–30 fps (target)	Lower resolution may be used to meet inference latency target
AI Inference Latency	< 3 s / frame (target)	Using INT8 quantized model on Hailo-8L NPU
Total Inspection Cycle	< 10 s (target)	Card scan → AI inference → gate decision
Supply Voltage	5 V (RPi), 5–6 V (Servo)	Regulated from LiPo; exact servo voltage TBD

3.4 External Libraries and SDKs

Library / SDK	Purpose	License
Raspberry Pi OS (64-bit, Bookworm)	Operating system for RPi 5	Free
Python 3.11+	Primary development language for all RPi modules	Open Source
Ultralytics / YOLO	Object detection model training framework	AGPL-3.0
ONNX Runtime	Optimised model inference on RPi (INT8)	Open Source
OpenCV	Camera capture and image preprocessing pipeline	Open Source
gpiozero / RPi.GPIO	GPIO pin control: RFID, servo motors, display	Open Source
Node.js (with Express.js framework)	REST API Backend Service	Open Source
PostgreSQL	Relational database for backend data layer	Open Source
React / HTML+JS or PyQt	Admin Panel and Turnstile Display frontend	Open Source
Git / GitHub	Version control and team collaboration	Free (student)

4. Inter-Module Communication Description

The table below describes every data exchange between module pairs. Struct and enum type names are left blank (TBD) pending agreement between module owners before coding begins. Frequency and direction are derived from the system flow diagram and state machine.

From	To	Data / Signal	Type / Protocol	Dir.	Notes / Frequency
MOD-03 (IoT)	MOD-01 (AI)	CameraFrame	dataclass (RGB888)	03 → 01	Raw image buffer passed to inference function.
MOD-01 (AI)	MOD-03 (IoT)	DetectionResult	dataclass (in-process return)	01 → 03	List of detected PPE classes and confidence scores. Consumed by IoT to compare against required PPE list.
MOD-03 (IoT)	MOD-02 (Turnstile)	gate_open()	gate_direction_t	03 → 02	gate.open() on GRANTED; gate.close() after passage timeout.
MOD-03 (IoT)	MOD-04 (Backend)	RFID UID lookup request: uid	string / HTTP GET /api/workers/card/{uid}	03 → 04	Triggered on each card scan. Returns worker profile and required_ppe list.
MOD-04 (Backend)	MOD-03 (IoT)	Card lookup response: worker + required_ppe OR 404 error	JSON / HTTP response	04 → 03	Real-time response in inspection flow step 2.
MOD-03 (IoT)	MOD-04 (Backend)	Inspection result write: worker_id, rfid_uid_scanned, result, detections, inspection_time_ms, camera_snapshot_url	JSON / HTTP POST /api/entry-logs	03 → 04	Triggered after PPE detection result is finalized.
MOD-04 (Backend)	MOD-03 (IoT)	Entry log creation ack: entry_log_id, result, scanned_at, missing_ppe	JSON / HTTP response	04 → 03	Returned per inspection submission.
MOD-05 (UI/UX)	MOD-04 (Backend)	Worker list query: is_active, role_id	query params / HTTP GET /api/workers	05 → 04	Used on worker management page load/filter actions.
MOD-05 (UI/UX)	MOD-04 (Backend)	Create worker payload: full_name, rfid_card_uid, role_id, photo_url	JSON / HTTP POST /api/workers	05 → 04	Used in RFID registration flow and manual worker creation.
MOD-05 (UI/UX)	MOD-04 (Backend)	Get worker by id	path param / HTTP GET /api/workers/{id}	05 → 04	Used for detail/edit screens.

MOD-05 (UI/UX)	MOD-04 (Backend)	Update worker payload	JSON / HTTP PUT /api/workers/{id}	05 -> 04	Used when editing worker profile/card/role.
MOD-05 (UI/UX)	MOD-04 (Backend)	Soft-delete worker request	HTTP DELETE /api/workers/{id}	05 -> 04	Used for deactivation instead of hard delete.
MOD-05 (UI/UX)	MOD-04 (Backend)	Role list request	HTTP GET /api/roles	05 -> 04	Used in role dropdowns and role management.
MOD-05 (UI/UX)	MOD-04 (Backend)	Create role payload: role_name, description	JSON / HTTP POST /api/roles	05 -> 04	Used by admin role creation.
MOD-05 (UI/UX)	MOD-04 (Backend)	Get role PPE requirement by role id	HTTP GET /api/roles/{id}/ppe	05 -> 04	Used in role PPE config screen.
MOD-05 (UI/UX)	MOD-04 (Backend)	Replace role PPE list: ppe_item_ids[]	JSON / HTTP PUT /api/roles/{id}/ppe	05 -> 04	Replace operation; submitted list overwrites existing set.
MOD-05 (UI/UX)	MOD-04 (Backend)	PPE catalog request	HTTP GET /api/ppe-items	05 -> 04	Used to populate assignable PPE items.
MOD-05 (UI/UX)	MOD-04 (Backend)	Entry log list filters: worker_id, result, start_date, end_date, limit, offset	query params / HTTP GET /api/entry-logs	05 -> 04	Used in audit/history/report pages.
MOD-05 (UI/UX)	MOD-04 (Backend)	Compliance stats filters: start_date, end_date	query params / HTTP GET /api/entry-logs/stats	05 -> 04	Used in dashboard cards/charts.
MOD-04 (Backend)	MOD-05 (UI/UX)	Stats response: total_scans, passed, failed, unknown_cards, compliance_rate, daily_data, most_missed_ppe, period	JSON / HTTP response	04 -> 05	daily_data supports date-based trend charting in selected range.
MOD-04 (Backend)	MOD-05 (UI/UX)	Worker list response: Worker[], total	JSON / HTTP response	04 -> 05	Returns active workers; frontend filters by name/RFID in JavaScript
MOD-04 (Backend)	MOD-05 (UI/UX)	Worker creation response: Worker (201)	JSON / HTTP response	04 -> 05	Returns created worker; 409 if RFID duplicate, 422 if field missing

MOD-04 (Backend)	MOD-05 (UI/UX)	Role PPE response: { role_id, role_name, ppe_items[] }	JSON / HTTP response	04 -> 05	Returns Ppeltem objects (id, item_key, display_name, icon_name)
MOD-04 (Backend)	MOD-05 (UI/UX)	Entry log list response: EntryLog[], total, limit, offset	JSON / HTTP response	04 -> 05	Paginated; sorted newest first
MOD-03 (IoT), MOD-05 (UI/UX)	MOD-04 (Backend)	Health check request	HTTP GET /api/health	03/05 -> 04	Optional liveness check before critical operations.
MOD-04 (Backend)	MOD-03 (IoT), MOD-05 (UI/UX)	Health response: status, timestamp	JSON / HTTP response	04 -> 03/05	Used for service monitoring and startup checks.

5. Known Risks & Open Questions

The following items represent interface decisions that are currently uncertain, hardware items not yet fully confirmed, or timing / resource constraints that may require renegotiation as development progresses.

ID	Risk / Open Question	Impact	Mitigation
R-01	Interface data type names (struct/enum) for all inter-module boundaries are TBD. All module owners must agree and document final names before Week 2 ends.	High	Hold a dedicated interface design meeting in Week 1. Freeze all type names and update this document to v0.2 before coding starts.
R-02	AI inference latency on Raspberry Pi 5 with AI Kit may exceed the 10-second cycle time target if the model is not sufficiently optimized.	High	Apply INT8 quantisation and use the Hailo-8L NPU via ONNX Runtime. Reduce input resolution if needed. Benchmark in Week 4 and fall back to a lighter model variant if target is not met.
R-03	PPE detection accuracy may degrade under variable indoor lighting conditions not covered by the training dataset.	Med	Apply photometric augmentation (brightness, contrast, noise) during training. Fine-tune on images captured in the actual deployment environment if possible.
R-04	Display hardware type (HDMI monitor vs. tablet vs. DSI touchscreen) has not been finalized, affecting the UI/UX Module implementation interface.	Med	Confirm display hardware selection by end of Week 1. UI/UX Module should design for the confirmed interface; WebSocket protocol is display-agnostic.
R-05	SPI clock frequency and exact GPIO pin assignments for RC522 RFID reader and servo PWM are not yet defined, which may cause GPIO conflicts between modules.	Med	IoT Module (MOD-03) to produce a definitive GPIO pin allocation table and SPI/PWM configuration before any wiring begins (Phase 1 deliverable).
R-06	Servo motor torque rating has not been verified against the final gate arm mechanical load. Insufficient torque would prevent reliable gate operation.	Low	Verify torque requirement against CAD model weight estimates in Phase 1. Select servo with a 20% torque margin; test open/close under load in Week 6.
R-07	Backend server Wi-Fi reachability from Raspberry Pi may be intermittent on the university network, breaking the card-lookup flow.	Low	Implement HTTP retry logic with exponential back-off on the IoT Module. Use a dedicated local Wi-Fi router for demo day to avoid shared-network interference.
R-08	PPE label names from MOD-01 have not been mapped to MOD-04 item_key strings (e.g. hard_hat, safety_vest). A mismatch causes every inspection to silently return FAIL.	High	MOD-01, MOD-03, and MOD-04 teams must agree on a shared PPE label convention before database seeding. Either enforce exact name matching or define a normalization table referenced by all three modules.
R-09	MOD-04 uses auto-increment database integers for PPE items (ppe_item_id, 1..N); MOD-01 uses PPEClass enum integers (0..10). These are different ID spaces. Passing a PPEClass value directly as a ppe_item_id silently references the wrong database record.	High	MOD-03 should send item_key strings in the detection payload; MOD-04 resolves the corresponding ppe_item_id server-side. Agree and document this before the integration build.

*This document is version **v0.2**. Changes will be communicated via the team GitHub repository and the weekly stand-up meeting.*